CampusPrint

University print desk simulator

Amir Baiseitov · SE-2505

Software Design Patterns · Final project defense · November 2026

Introduction

Two students bring work to a print desk. One wants a bound thesis. The other needs two copies of a short application. The operator must quote each order, choose a job to print and record what actually finished. CampusPrint models those decisions in one Java application.

The interesting case occurs when the operator changes the queue policy after accepting both jobs. Arrival order favors the thesis. Shortest job first favors the application, until the requested copy count makes it the larger job. The decision depends on the same print volume used for billing and statistics.

The reference orders

Order	Selected pages	Copies	Services	Amount
1 · Thesis	1–80 of 80	1	Print, cover, bind	1,900 KZT
2 · Application	2–3 of 5	2	Print	80 KZT

Scope and contribution

I implemented the six course patterns, integrated them in a single print workflow and prepared its verification cases. The simulation runs in one process with an in-memory queue. A successful printer call writes to the terminal. An injected paper jam exercises the failure path. The program calculates every quoted amount, queue selection and completion count.

Architecture

The two gray boxes connect the six pattern groups. `Main` assembles the demonstration. `PrintOrder` pairs a numbered job with its pricing services. Both document types read that same order.

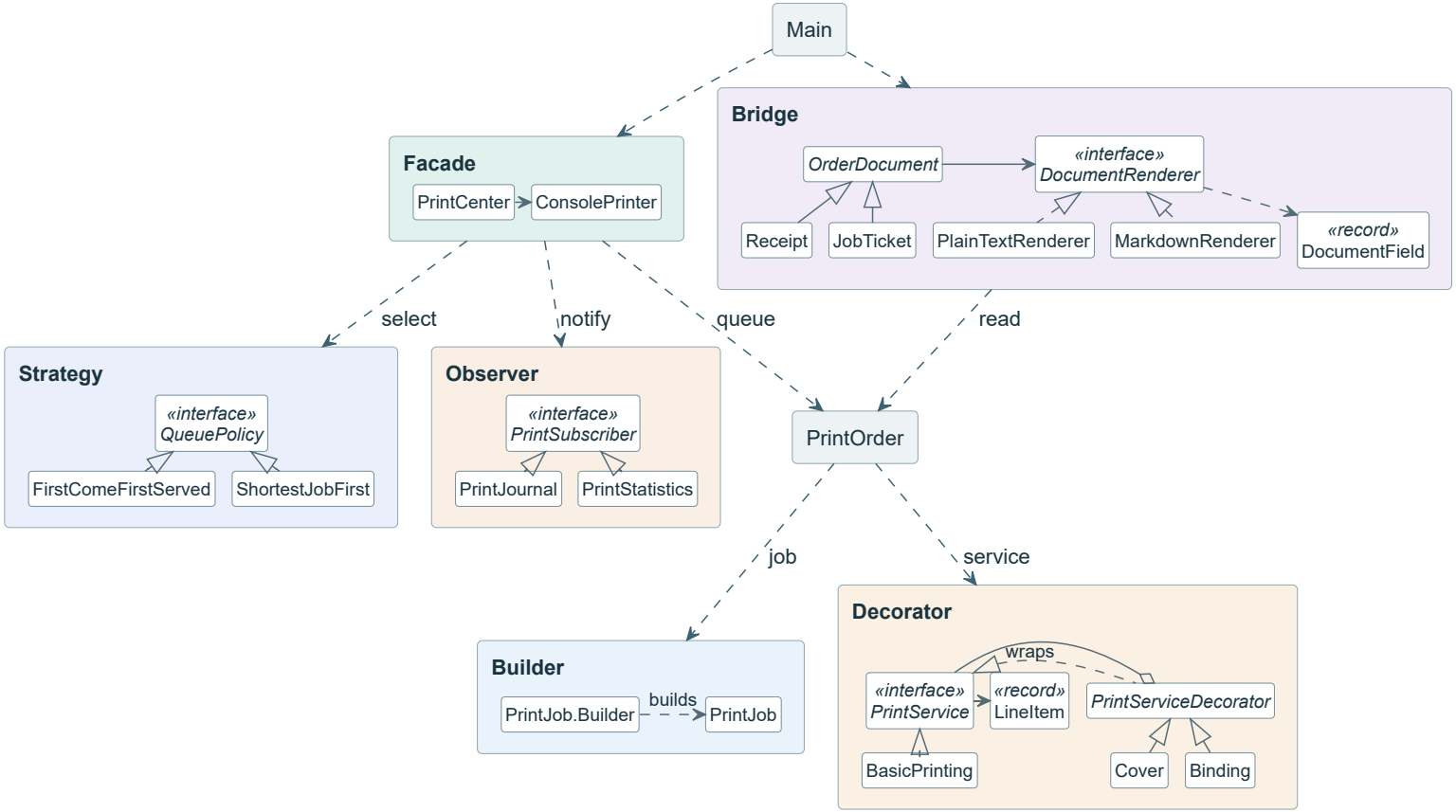


Figure 1. CampusPrint class overview. Within each group, hollow triangles point to a supertype. Labeled dependencies between groups show how the parts collaborate.

Follow the **queue** relationship to see what the operator schedules. Follow **service** to find where a charge originates. Follow **read** from Bridge to see how that charge reaches the customer.

Building and pricing an order

Builder

`PrintJob.Builder` starts with a document name and its total page count. The full document and one copy are the defaults. Range and copy settings remain private to the builder until `build()` validates them and takes an immutable snapshot. An invalid job cannot enter the queue.

```
PrintJob application = new PrintJob.Builder("Application", 5)
        .range(2, 3)
        .copies(2)
        .build();
```

Both endpoints count. Pages 2 through 3 produce two pages per copy, or four printed pages for this order. The builder checks the multiplication in 64-bit arithmetic before accepting a volume that must fit a Java `int`. This keeps an oversized batch from appearing to cost zero or jumping ahead in the queue.

Decorator

The operator chooses finishing services at acceptance. Each decorator wraps a `PrintService`, preserves its line items and appends one charge of its own. The chain rejects a second occurrence of the same finish.

```
PrintJob thesis = new PrintJob.Builder("Thesis", 80).build();
PrintOrder order = center.acceptOrder(thesis,
        new Binding(new Cover(new BasicPrinting())));
```

Thesis receipt	Calculation	Amount
Printing	80 printed pages × 20 KZT	1,600 KZT
Cover	1 copy × 100 KZT	100 KZT
Binding	1 copy × 200 KZT	200 KZT
Total	**Sum of the line items**	**1,900 KZT**

The production ticket takes its operation names from the same line items. Choosing binding once therefore supplies both the price on the receipt and the instruction at the printer. Charges use whole tenge stored in a Java `long`.

Scheduling and completion

Strategy. `FirstComeFirstServed` selects the queue head. `ShortestJobFirst` scans for the smallest print volume and keeps the earlier entry on a tie. Preview and print call the same selection method. Switching policies preserves arrival order.

Facade. One call to `PrintCenter.printNext()` selects the job, runs the printer, removes the completed order and publishes its completion. A paper jam interrupts this sequence before removal. An operator can clear the fault and retry that same order.

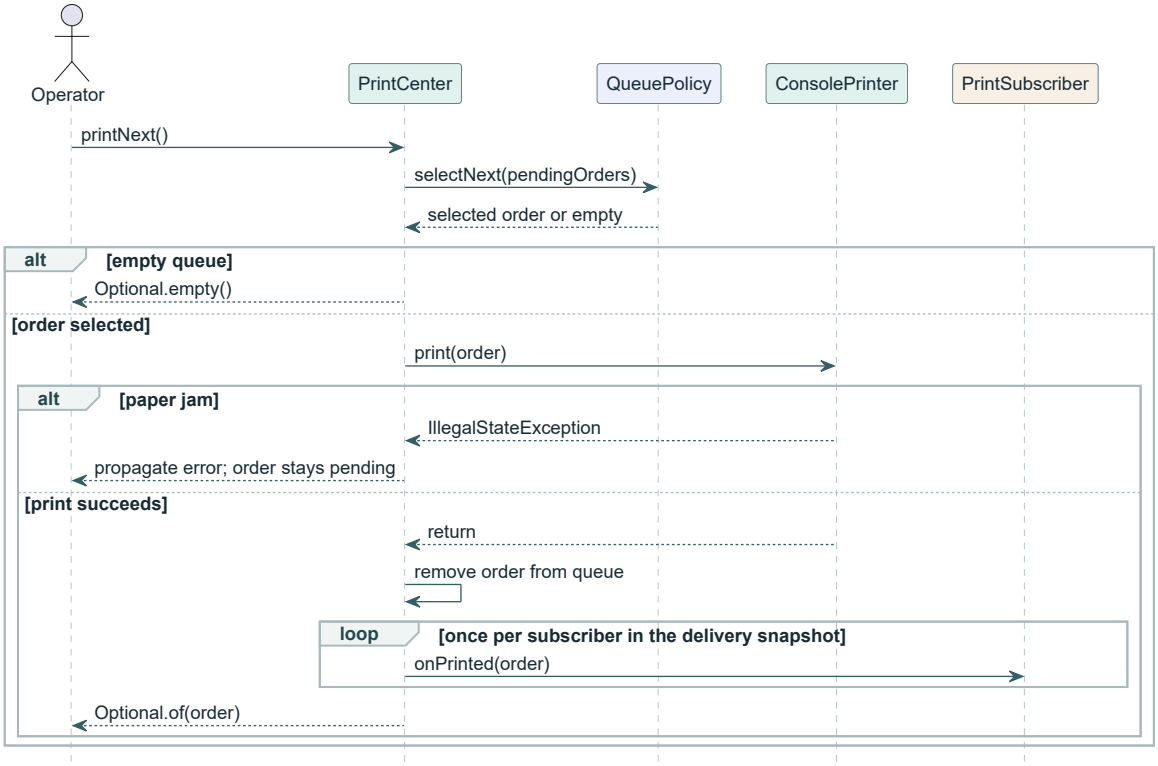


Figure 2. A print request has three outcomes. Only a successful return from the printer reaches the notification loop.

Observer. The journal records the order number and title. Statistics count completed jobs and pages. The center snapshots the subscriber list before dispatch, so a subscription change during a callback applies to the next event. Each registered observer receives one call for the current event.

Documents and output formats

Bridge. The customer needs an itemized price. The operator needs the range, copies and finishing instructions. `Receipt` and `JobTicket` choose the fields. Their shared base class holds a `DocumentRenderer` and passes it a title and those fields.

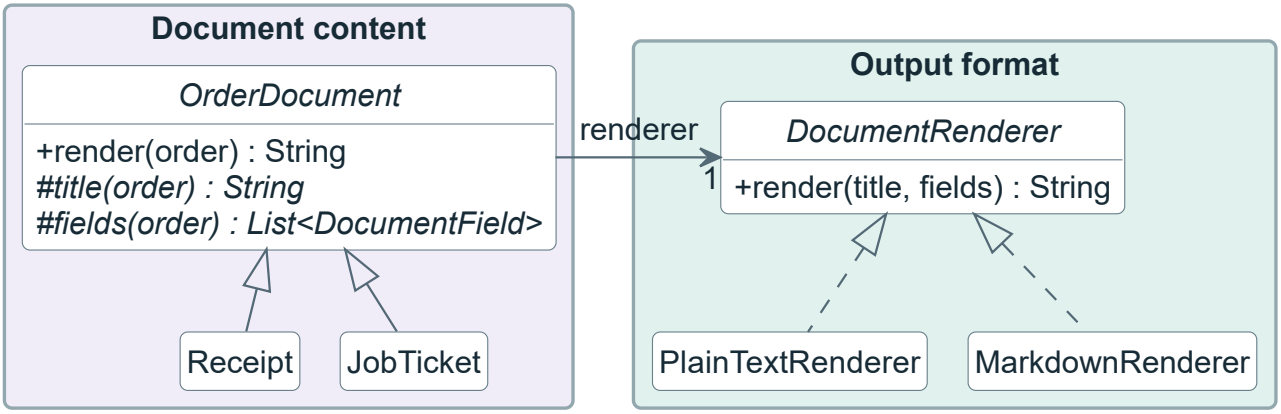


Figure 3. Document content and output format vary independently. Both concrete document types inherit the same renderer reference.

The text renderer aligns labels and values. The Markdown renderer emits a two-column table and escapes table delimiters in input. The demo renders both documents in both formats for order 1. Its price and finishing services agree across all four outputs.

```
Receipt #1 / Thesis
  Printing          1,600 KZT
  Cover             100 KZT
  Binding           200 KZT
  Total             1,900 KZT

Job ticket #1 / Thesis
  Document          Thesis
  Page range        1-80 of 80
  Copies            1
  Printed pages     80
  Operations        Printing, Cover, Binding
```

Figure 4. Two plain-text documents from the same accepted order. Cover and Binding appear on the receipt and in the operator's instructions.

Demonstration results

Switching policy on a live queue

FIFO previews order 1, which arrived first. The operator switches to shortest job first. The four-page application prints next. Its completion reaches both subscribers while the thesis remains pending.

```
CampusPrint / Print desk
#1 Thesis | copies: 1 | pages: 80 | 1,900 KZT
#2 Application | copies: 2 | pages: 4 | 80 KZT

First come, first served -> #1 Thesis (80 pages)
Shortest job first -> #2 Application (4 pages)
Printed #2 Application (4 pages)
Journal: #2 Application
Completed orders: 1 | Printed pages: 4
```

Figure 5. Terminal capture of the Java process showing the two previews and the first completed job.

Changing the order quantity

After the thesis prints, the empty-queue request leaves the totals at two orders and 84 pages. The next comparison starts with a fresh queue. Fifty copies of the two-page application now require 100 pages, so the 80-page thesis wins under the same shortest-job policy.

```
Printed #1 Thesis (80 pages)
Queue empty.
Journal: #2 Application, #1 Thesis
Completed orders: 2 | Printed pages: 84

Volume comparison / fresh queue
#1 Thesis | copies: 1 | pages: 80 | 1,900 KZT
#2 Application | copies: 50 | pages: 100 | 2,000 KZT
Shortest job first -> #1 Thesis (80 pages)
```

Figure 6. End of the original run and the fresh comparison. Order numbers restart with each new print center.

To change the comparison at the defense, run `java -cp out campusprint.Main 10`. Ten copies require 20 pages and cost 400 KZT. At 40 copies the volumes tie, so arrival order decides.

Verification and conclusions

Verification

The project compiles against the Java 17 API. The executable checks use the standard library and throw an assertion error on a mismatch. Run `java -cp out campusprint.PrintCenterChecks` after compilation.

Case	Expected and observed result
Invalid job	Reject a blank title, zero copies, an inverted range and an out-of-bounds range.
Volume limit	Reject 65,536 pages × 65,536 copies before accepting the job. Large valid charges and cumulative page totals retain their full value.
Finishing	Three copies of four pages with two finishes total 1,140 KZT. Reject a duplicate cover even when binding is nested between the two covers.
Scheduling	On a tie the earlier job wins. Preview leaves both entries in place.
Paper jam and retry	The queue and journal retain their pre-print state. After recovery, each job completes once.
Subscription changes	A subscriber can remove itself during delivery. The next subscriber still sees that print. A newly registered subscriber starts with the next print.
Document formatting	A title containing a vertical bar and a line break stays in one Markdown table cell.

Conclusion

A single order carries its validated page range, copy count and chosen services from acceptance through billing, scheduling and reporting. In the reference run the print center completes 84 pages in two jobs. Raising the application quantity reverses the scheduling decision without any change to the algorithm.

Future work

A persistent queue would let an operator resume outstanding work after a restart. It would need a durable order identifier and an explicit recovery policy for a job interrupted during printing. A further extension would route orders to printers with different finishing capabilities. The operator's ticket already identifies the required operations.

Java source

The PDF appendix contains the complete application listing from `src/campusprint`. The source ZIP also includes the executable regression checks, the editable UML and the build instructions.

Appendix · Java source

campusprint/Main.java

```
package campusprint;

import campusprint.center.PrintCenter;
import campusprint.document.JobTicket;
import campusprint.document.MarkdownRenderer;
import campusprint.document.PlainTextRenderer;
import campusprint.document.Receipt;
import campusprint.domain.PrintJob;
import campusprint.domain.PrintOrder;
import campusprint.observer.PrintJournal;
import campusprint.observer.PrintStatistics;
import campusprint.pricing.BasicPrinting;
import campusprint.pricing.Binding;
import campusprint.pricing.Cover;
import campusprint.queue.ShortestJobFirst;

public final class Main {
    public static void main(String[] args) {
        try {
            int comparisonCopies = args.length == 0 ? 50 : Integer.parseInt(args[0]);
            if (args.length > 1 || comparisonCopies < 1) {
                throw new IllegalArgumentException("Expected one positive copy count.");
            }
            PrintJob comparisonJob = application(comparisonCopies);
            runPrintDesk();
            compareVolume(comparisonJob);
        } catch (IllegalArgumentException invalid) {
            System.err.println(invalid.getMessage());
            System.err.println("Usage: java -cp out campusprint.Main [comparisonCopies]");
            System.exit(2);
        }
    }

    private static PrintOrder acceptThesis(PrintCenter center) {
        PrintJob thesis = new PrintJob.Builder("Thesis", 80).build();
        return center.acceptOrder(thesis, new Binding(new Cover(new BasicPrinting())));
    }

    private static PrintJob application(int copies) {
        return new PrintJob.Builder("Application", 5).range(2, 3).copies(copies).build();
    }

    private static void runPrintDesk() {
        PrintCenter center = new PrintCenter();
        PrintJournal journal = new PrintJournal();
        PrintStatistics statistics = new PrintStatistics();
        center.subscribe(journal);
        center.subscribe(statistics);
```

Appendix · Java source

campusprint/Main.java (continued)

```
        System.out.println("CampusPrint / Print desk");
        PrintOrder thesis = acceptThesis(center);
        PrintOrder application = center.acceptOrder(application(2), new BasicPrinting());
        System.out.println(thesis);
        System.out.println(application);
        System.out.println();
        showPreview(center);
        center.setQueuePolicy(new ShortestJobFirst());
        showPreview(center);
        center.printNext();
        System.out.println(journal.summary());
        System.out.println(statistics.summary());

        PlainTextRenderer text = new PlainTextRenderer();
        MarkdownRenderer markdown = new MarkdownRenderer();
        System.out.println();
        System.out.println(new Receipt(text).render(thesis));
        System.out.println();
        System.out.println(new JobTicket(text).render(thesis));
        System.out.println();
        System.out.println(new Receipt(markdown).render(thesis));
        System.out.println();
        System.out.println(new JobTicket(markdown).render(thesis));
        System.out.println();

        center.printNext();
        center.printNext();
        System.out.println(journal.summary());
        System.out.println(statistics.summary());
    }

    private static void compareVolume(PrintJob application) {
        PrintCenter center = new PrintCenter();
        PrintOrder thesis = acceptThesis(center);
        PrintOrder other = center.acceptOrder(application, new BasicPrinting());
        center.setQueuePolicy(new ShortestJobFirst());
        System.out.println();
        System.out.println("Volume comparison / fresh queue");
        System.out.println(thesis);
        System.out.println(other);
        showPreview(center);
    }

    private static void showPreview(PrintCenter center) {
        PrintOrder next = center.peekNext().orElseThrow();
        System.out.printf("%s -> #%d %s (%d pages)%n",
                center.queuePolicy().name(), next.id(), next.job().documentName(),
next.printedPages());
    }
}
```

Appendix · Java source

campusprint/center/ConsolePrinter.java

```
package campusprint.center;

import campusprint.domain.PrintOrder;

public final class ConsolePrinter {
    private boolean simulateFailure;

    public void setSimulateFailure(boolean simulateFailure) {
        this.simulateFailure = simulateFailure;
    }

    public void print(PrintOrder order) {
        if (simulateFailure) {
            throw new IllegalStateException("Paper jam on order #" + order.id() + ".");
        }
        System.out.printf("Printed #%d %s (%d pages)%n",
                order.id(), order.job().documentName(), order.printedPages());
    }
}
```

campusprint/center/PrintCenter.java

```
package campusprint.center;

import campusprint.domain.PrintJob;
import campusprint.domain.PrintOrder;
import campusprint.observer.PrintSubscriber;
import campusprint.pricing.PrintService;
import campusprint.queue.FirstComeFirstServed;
import campusprint.queue.QueuePolicy;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.Optional;

public final class PrintCenter {
    private final List<PrintOrder> pendingOrders = new ArrayList<>();
    private final List<PrintSubscriber> subscribers = new ArrayList<>();
    private final ConsolePrinter printer;
    private QueuePolicy queuePolicy;
    private int nextOrderId = 1;

    public PrintCenter() {
        this(new ConsolePrinter(), new FirstComeFirstServed());
    }

    public PrintCenter(ConsolePrinter printer, QueuePolicy initialPolicy) {
        this.printer = Objects.requireNonNull(printer, "printer");
        this.queuePolicy = Objects.requireNonNull(initialPolicy, "initialPolicy");
    }
```

Appendix · Java source

campusprint/center/PrintCenter.java (continued)

```
    public void subscribe(PrintSubscriber subscriber) {
        Objects.requireNonNull(subscriber, "subscriber");
        if (!subscribers.contains(subscriber)) {
            subscribers.add(subscriber);
        }
    }

    public void unsubscribe(PrintSubscriber subscriber) {
        subscribers.remove(subscriber);
    }

    public PrintOrder acceptOrder(PrintJob job, PrintService service) {
        PrintOrder order = new PrintOrder(nextOrderId, job, service);
        pendingOrders.add(order);
        nextOrderId++;
        return order;
    }

    public void setQueuePolicy(QueuePolicy policy) {
        queuePolicy = Objects.requireNonNull(policy, "policy");
    }

    public QueuePolicy queuePolicy() {
        return queuePolicy;
    }

    public Optional<PrintOrder> peekNext() {
        return queuePolicy.selectNext(pendingOrders());
    }

    public List<PrintOrder> pendingOrders() {
        return List.copyOf(pendingOrders);
    }

    public Optional<PrintOrder> printNext() {
        Optional<PrintOrder> next = peekNext();
        if (next.isEmpty()) {
            System.out.println("Queue empty.");
            return next;
        }
        PrintOrder order = next.orElseThrow();
        printer.print(order);
        pendingOrders.remove(order);
        notifySubscribers(order);
        return next;
    }
```

Appendix · Java source

campusprint/center/PrintCenter.java (continued)

```
    private void notifySubscribers(PrintOrder order) {
        // Subscriptions changed during delivery take effect on the next print.
        for (PrintSubscriber subscriber : List.copyOf(subscribers)) {
            try {
                subscriber.onPrinted(order);
            } catch (RuntimeException failure) {
                System.err.printf("Notification failed for order #%d: %s%n", order.id(),
failure.getMessage());
            }
        }
    }
}
```

campusprint/document/DocumentField.java

```
package campusprint.document;

import java.util.Objects;

public record DocumentField(String label, String value) {
    public DocumentField {
        if (label == null || label.isBlank()) {
            throw new IllegalArgumentException("A document field needs a label.");
        }
        Objects.requireNonNull(value, "value");
    }
}
```

campusprint/document/DocumentRenderer.java

```
package campusprint.document;

import java.util.List;

public interface DocumentRenderer {
    String render(String title, List<DocumentField> fields);
}
```

campusprint/document/JobTicket.java

```
package campusprint.document;

import campusprint.domain.PrintJob;
import campusprint.domain.PrintOrder;
import campusprint.pricing.LineItem;
import java.util.List;
import java.util.stream.Collectors;

public final class JobTicket extends OrderDocument {
    public JobTicket(DocumentRenderer renderer) {
        super(renderer);
    }
```

Appendix · Java source

campusprint/document/JobTicket.java (continued)

```
    @Override
    protected String title(PrintOrder order) {
        return "Job ticket #" + order.id() + " / " + order.job().documentName();
    }

    @Override
    protected List<DocumentField> fields(PrintOrder order) {
        PrintJob job = order.job();
        String operations =
order.items().stream().map(LineItem::name).collect(Collectors.joining(", "));
        return List.of(
                new DocumentField("Document", job.documentName()),
                new DocumentField("Page range", "%d-%d of %d".formatted(job.fromPage(),
job.toPage(), job.totalPages())),
                new DocumentField("Copies", Integer.toString(job.copies())),
                new DocumentField("Printed pages", Integer.toString(job.printedPages())),
                new DocumentField("Operations", operations));
    }
}
```

campusprint/document/MarkdownRenderer.java

```
package campusprint.document;

import java.util.List;

public final class MarkdownRenderer implements DocumentRenderer {
    @Override
    public String render(String title, List<DocumentField> fields) {
        StringBuilder table = new StringBuilder("### ").append(escape(title));
        table.append("\n\n| Field | Value |\n| --- | --- |");
        for (DocumentField field : fields) {
            table.append("\n| ").append(escape(field.label()))
                    .append(" | ").append(escape(field.value())).append(" |");
        }
        return table.toString();
    }

    private static String escape(String text) {
        return text.replace("\\", "\\\\").replace("|", "\\|").replaceAll("\\R", " ")
                .replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
                .replace("*", "\\*").replace("_", "\\_").replace("`", "\\`")
                .replace("[", "\\[").replace("]", "\\]").replace("~", "\\~");
    }
}
```

Appendix · Java source

campusprint/document/OrderDocument.java

```
package campusprint.document;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Objects;

public abstract class OrderDocument {
    private final DocumentRenderer renderer;

    protected OrderDocument(DocumentRenderer renderer) {
        this.renderer = Objects.requireNonNull(renderer, "renderer");
    }

    protected abstract String title(PrintOrder order);

    protected abstract List<DocumentField> fields(PrintOrder order);

    public final String render(PrintOrder order) {
        return renderer.render(title(order), fields(order));
    }
}
```

campusprint/document/PlainTextRenderer.java

```
package campusprint.document;

import java.util.List;

public final class PlainTextRenderer implements DocumentRenderer {
    @Override
    public String render(String title, List<DocumentField> fields) {
        StringBuilder text = new StringBuilder(title);
        for (DocumentField field : fields) {
            text.append(System.lineSeparator())
                    .append("  %-16s %s".formatted(field.label(), field.value()));
        }
        return text.toString();
    }
}
```

campusprint/document/Receipt.java

```
package campusprint.document;

import campusprint.domain.PrintOrder;
import campusprint.pricing.LineItem;
import java.util.ArrayList;
import java.util.List;

public final class Receipt extends OrderDocument {
    public Receipt(DocumentRenderer renderer) {
        super(renderer);
    }
```

Appendix · Java source

campusprint/document/Receipt.java (continued)

```
    @Override
    protected String title(PrintOrder order) {
        return "Receipt #" + order.id() + " / " + order.job().documentName();
    }

    @Override
    protected List<DocumentField> fields(PrintOrder order) {
        List<DocumentField> fields = new ArrayList<>();
        for (LineItem item : order.items()) {
            fields.add(new DocumentField(item.name(), LineItem.formatTenge(item.amount())));
        }
        fields.add(new DocumentField("Total", LineItem.formatTenge(order.totalCost())));
        return fields;
    }
}
```

campusprint/domain/PrintJob.java

```
package campusprint.domain;

public final class PrintJob {
    private final String documentName;
    private final int totalPages;
    private final int fromPage;
    private final int toPage;
    private final int copies;

    private PrintJob(Builder builder) {
        documentName = builder.documentName;
        totalPages = builder.totalPages;
        fromPage = builder.fromPage;
        toPage = builder.toPage;
        copies = builder.copies;
    }

    public String documentName() {
        return documentName;
    }

    public int totalPages() {
        return totalPages;
    }

    public int fromPage() {
        return fromPage;
    }

    public int toPage() {
        return toPage;
    }

    public int copies() {
        return copies;
    }
```

Appendix · Java source

campusprint/domain/PrintJob.java (continued)

```
    public int pagesPerCopy() {
        return toPage - fromPage + 1;
    }

    public int printedPages() {
        return pagesPerCopy() * copies;
    }

    public static final class Builder {
        private final String documentName;
        private final int totalPages;
        private int fromPage = 1;
        private int toPage;
        private int copies = 1;

        public Builder(String documentName, int totalPages) {
            this.documentName = documentName;
            this.totalPages = totalPages;
            this.toPage = totalPages;
        }

        public Builder range(int from, int to) {
            fromPage = from;
            toPage = to;
            return this;
        }

        public Builder copies(int count) {
            copies = count;
            return this;
        }

        public PrintJob build() {
            if (documentName == null || documentName.isBlank()) {
                throw new IllegalArgumentException("Enter a document name.");
            }
            if (totalPages < 1 || copies < 1) {
                throw new IllegalArgumentException("Page and copy counts must be positive.");
            }
            if (fromPage < 1 || toPage < fromPage || toPage > totalPages) {
                throw new IllegalArgumentException("Select a range within pages 1-" + totalPages +
"."");
            }
            if ((long) (toPage - fromPage + 1) * copies > Integer.MAX_VALUE) {
                throw new IllegalArgumentException("Print volume exceeds the per-job page
limit.");
            }
            return new PrintJob(this);
        }
    }
}
```

Appendix · Java source

campusprint/domain/PrintOrder.java

```
package campusprint.domain;

import campusprint.pricing.LineItem;
import campusprint.pricing.PrintService;
import java.util.List;
import java.util.Objects;

public final class PrintOrder {
    private final int id;
    private final PrintJob job;
    private final PrintService service;

    public PrintOrder(int id, PrintJob job, PrintService service) {
        if (id <= 0) {
            throw new IllegalArgumentException("Order IDs must be positive.");
        }
        this.id = id;
        this.job = Objects.requireNonNull(job, "job");
        this.service = Objects.requireNonNull(service, "service");
    }

    public int id() {
        return id;
    }

    public PrintJob job() {
        return job;
    }

    public PrintService service() {
        return service;
    }

    public int printedPages() {
        return job.printedPages();
    }

    public List<LineItem> items() {
        return List.copyOf(service.itemsFor(job));
    }

    public long totalCost() {
        return items().stream().mapToLong(LineItem::amount).sum();
    }

    @Override
    public String toString() {
        return "#%d %s | copies: %d | pages: %d | %s".formatted(
                id, job.documentName(), job.copies(), printedPages(),
LineItem.formatTenge(totalCost()));
    }
}
```

Appendix · Java source

campusprint/observer/PrintJournal.java

```
package campusprint.observer;

import campusprint.domain.PrintOrder;
import java.util.ArrayList;
import java.util.List;

public final class PrintJournal implements PrintSubscriber {
    private final List<String> entries = new ArrayList<>();

    @Override
    public void onPrinted(PrintOrder order) {
        entries.add("#" + order.id() + " " + order.job().documentName());
    }

    public List<String> entries() {
        return List.copyOf(entries);
    }

    public String summary() {
        return "Journal: " + (entries.isEmpty() ? "(empty)" : String.join(", ", entries));
    }
}
```

campusprint/observer/PrintStatistics.java

```
package campusprint.observer;

import campusprint.domain.PrintOrder;

public final class PrintStatistics implements PrintSubscriber {
    private int completedOrders;
    private long totalPrintedPages;

    @Override
    public void onPrinted(PrintOrder order) {
        completedOrders++;
        totalPrintedPages += order.printedPages();
    }

    public int completedOrders() {
        return completedOrders;
    }

    public long totalPrintedPages() {
        return totalPrintedPages;
    }

    public String summary() {
        return "Completed orders: %d | Printed pages: %d".formatted(completedOrders,
totalPrintedPages);
    }
}
```

Appendix · Java source

campusprint/observer/PrintSubscriber.java

```
package campusprint.observer;

import campusprint.domain.PrintOrder;

@FunctionalInterface
public interface PrintSubscriber {
    void onPrinted(PrintOrder order);
}
```

campusprint/pricing/BasicPrinting.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;
import java.util.List;

public final class BasicPrinting implements PrintService {
    public static final int PRICE_PER_PAGE = 20;
    public static final String SERVICE_NAME = "Printing";

    @Override
    public List<LineItem> itemsFor(PrintJob job) {
        return List.of(new LineItem(SERVICE_NAME, (long) job.printedPages() * PRICE_PER_PAGE));
    }
}
```

campusprint/pricing/Binding.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;

public final class Binding extends PrintServiceDecorator {
    public static final int PRICE_PER_COPY = 200;
    public static final String SERVICE_NAME = "Binding";

    public Binding(PrintService wrapped) {
        super(wrapped, SERVICE_NAME);
    }

    @Override
    protected long costFor(PrintJob job) {
        return (long) job.copies() * PRICE_PER_COPY;
    }
}
```

Appendix · Java source

campusprint/pricing/Cover.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;

public final class Cover extends PrintServiceDecorator {
    public static final int PRICE_PER_COPY = 100;
    public static final String SERVICE_NAME = "Cover";

    public Cover(PrintService wrapped) {
        super(wrapped, SERVICE_NAME);
    }

    @Override
    protected long costFor(PrintJob job) {
        return (long) job.copies() * PRICE_PER_COPY;
    }
}
```

campusprint/pricing/LineItem.java

```
package campusprint.pricing;

import java.util.Locale;

public record LineItem(String name, long amount) {
    public LineItem {
        if (name == null || name.isBlank() || amount < 0) {
            throw new IllegalArgumentException("Each charge needs a name and a nonnegative
amount.");
        }
    }

    public static String formatTenge(long amount) {
        return String.format(Locale.ROOT, "%,d KZT", amount);
    }
}
```

campusprint/pricing/PrintService.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;
import java.util.List;

public interface PrintService {
    List<LineItem> itemsFor(PrintJob job);
}
```

Appendix · Java source

campusprint/pricing/PrintServiceDecorator.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public abstract class PrintServiceDecorator implements PrintService {
    private final PrintService wrapped;
    private final String name;

    protected PrintServiceDecorator(PrintService wrapped, String name) {
        this.wrapped = Objects.requireNonNull(wrapped, "wrapped");
        this.name = name;
        PrintService current = wrapped;
        while (current instanceof PrintServiceDecorator finish) {
            if (finish.name.equals(name)) {
                throw new IllegalArgumentException(name + " is already included.");
            }
            current = finish.wrapped;
        }
    }

    protected abstract long costFor(PrintJob job);

    @Override
    public final List<LineItem> itemsFor(PrintJob job) {
        List<LineItem> items = new ArrayList<>(wrapped.itemsFor(job));
        items.add(new LineItem(name, costFor(job)));
        return List.copyOf(items);
    }
}
```

campusprint/queue/FirstComeFirstServed.java

```
package campusprint.queue;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Optional;

public final class FirstComeFirstServed implements QueuePolicy {
    @Override
    public Optional<PrintOrder> selectNext(List<PrintOrder> pendingOrders) {
        if (pendingOrders == null || pendingOrders.isEmpty()) {
            return Optional.empty();
        }
        return Optional.of(pendingOrders.get(0));
    }

    @Override
    public String name() {
        return "First come, first served";
    }
}
```

Appendix · Java source

campusprint/queue/QueuePolicy.java

```
package campusprint.queue;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Optional;

public interface QueuePolicy {
    /** The input is in arrival order. Return empty only if the queue is empty. */
    Optional<PrintOrder> selectNext(List<PrintOrder> pendingOrders);

    String name();
}
```

campusprint/queue/ShortestJobFirst.java

```
package campusprint.queue;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Optional;

public final class ShortestJobFirst implements QueuePolicy {
    @Override
    public Optional<PrintOrder> selectNext(List<PrintOrder> pendingOrders) {
        if (pendingOrders == null || pendingOrders.isEmpty()) {
            return Optional.empty();
        }
        PrintOrder shortest = pendingOrders.get(0);
        for (PrintOrder order : pendingOrders) {
            if (order.printedPages() < shortest.printedPages()) {
                shortest = order;
            }
        }
        return Optional.of(shortest);
    }

    @Override
    public String name() {
        return "Shortest job first";
    }
}
```